

Load Dataset

```
In [73]: import pandas as pd
import json

# Load dataset and rename columns
df = pd.read_csv("data/cost-of-living.csv", index_col=0)
df.rename(columns=json.load(open("data/attribute_names.json", "r")), inplace=True)
```

Data Preperation

Schritte:

1. ☐ Filtern der Daten
 - ☐ Filtern nach Datensätzen
 - ☐ Anhand des `data_quality`-Attributs
 - ☐ Anhand einer eigenen Qualitätsmetrik?
 - ☐ Filtern nach Features
 - ☐ Filtern von Ausreißern
2. ☐ Einfügen von fehlenden Werten
3. ☐ Datenintegration
4. ☐ Datentransformation
 - ☐ z-Transformation?
 - ☐ Diskretisierung?

Filtern der Daten

Filtern nach expliziter Datenqualität

```
In [74]: df = df[df["data_quality"] == 1].copy() # Lösche alle schlechten Einträge
df.reset_index(drop=True, inplace=True)
df.drop(columns=["data_quality"], inplace=True)
```

Doppelte Städtenamen

```
In [75]: val_counts = df["city"].value_counts()
duplicates_city = val_counts[val_counts > 1].sum()
val_counts = df[["city", "country"]].value_counts()
duplicates_country = val_counts[val_counts > 1].sum()

print(f"Es kommen {duplicates_city} Städtenamen mehrfach vor, wenn man das L
```

Es kommen 16 Städtenamen mehrfach vor, wenn man das Land berücksichtigt sind davon 0 doppelt.

Der Kontinent der einzelnen Städte wird mithilfe der Python-bibliothek `pycountry-convert` ermittelt. Hierzu sind mehrere Zwischenschritte nötig:

1. Zuerst werden die Ländernamen in Länderkürzel gemäß [ISO 3166-1 alpha-2](#) umgewandelt,
2. diese werden wiederum in Kontinentkürzel umgewandelt.
3. Zu diesen können die Namen der Kontinente ermittelt werden und dies als Spalte des Dataframes gespeichert werden.

Weil die Information des Länderkürzels im späteren Verlauf noch von Interesse ist, wird der erste Schritt separat in der direkt folgenden Zelle abgehandelt und die Information in einer extra Spalte gespeichert. Die beiden anderen Schritte werden gemeinsam in der darauf folgenden Zelle implementiert.

Noch wichtig zu erwähnen sind eine handvoll Ausnahmen, die nicht von der Bibliothek abgehandelt werden. Diese wurden deshalb manuell recherchiert (siehe Artikel zu [ISO 3166-1 alpha-2](#)) und jeweils in Form eines dictionaries `exceptions` abgehandelt.

Anreichern der Daten

Die folgenden Zellen Stellen das Anreichern des Dataframes um Kontinent-, Breiten-/Längengrad- und Höheninformationen dar.

Kontinentinformationen

```
In [76]: import pycountry_convert as pc

# Umwandlung der Ländernamen in Länderkürzel
def country_name_to_alpha2(country_name):
    # Ausnahmen für Länder, die nicht in pycountry_convert enthalten sind
    exceptions = {
        "Bosnia And Herzegovina": "BA",
        "Trinidad And Tobago": "TT",
        "Isle Of Man": "IM",
        "Curacao": "CW",
        "Kosovo (Disputed Territory)": "XK",
    }
    if country_name in exceptions.keys():
        return exceptions[country_name]
    return pc.country_name_to_country_alpha2(country_name)

df.loc[:, "alpha2"] = df["country"].apply(country_name_to_alpha2)
```

```
In [77]: # Umwandlung der Länderkürzel in Kontinentkürzel und -namen
def alpha2_to_continent(alpha2):
    # Ausnahmen für Länder, die nicht in pycountry_convert enthalten sind
```

```

exceptions = {
    "BA": "Europe",
    "TT": "South America",
    "IM": "Europe",
    "CW": "South America",
    "XK": "Europe",
}
if alpha2 in exceptions.keys():
    return exceptions[alpha2]
continent_code = pc.country_alpha2_to_continent_code(alpha2)
continent_name = pc.convert_continent_code_to_continent_name(continent_code)
return continent_name

df.loc[:, "continent"] = df["alpha2"].apply(alpha2_to_continent)

```

Geographische Koordinaten

Die Ermittlung der geographischen Koordinaten dient nicht nur der Ermittlung der Höhenmeter, sondern erlaubt auch eine bessere Visualisierung und somit Exploration der Daten.

Sie umfasst folgende Schritte:

1. Laden und Vorverarbeiten der externen Datenquelle zu Städten und deren Informationen
2. Datenaggregation
 - A. über primären Städtenamen.
 - B. über alternative Städtenamen.
 - C. von Ausnahmen.

Der von [GeoNames](#) stammende Datensatz umfasst Städte mit mindestens 15000 Einwohnern oder Hauptstädte. Im ersten Schritt wird er geladen und die relevanten Zeilen mithilfe des Arguments `usecols=` ausgewählt.

```

In [78]: # Laden der Daten als Pandas DataFrame; Auswahl relevanter Spalten
cities = pd.read_table(
    "data/cities15000.txt",
    names=[
        "geonameid", "name", "asciiname", "alternatenames", "latitude", "lon
        "feature class", "feature code", "country code", "cc2", "admin1 code
        "admin3 code", "admin4 code", "population", "elevation", "dem", "tim
    ],
    usecols=["asciiname", "country code", "latitude", "longitude", "alternat
)

# Vorverarbeitung: Eindeutige Stadt<->Staat Paare
cities = (
    cities
    .groupby(["asciiname", "country code"])
    .first()
    .reset_index()
)

```

```
)
# Vorverarbeitung: Umwandlung der alternatenames-Information von String zu List
cities["alternatenames"] = cities["alternatenames"].apply(lambda x: x.split(
cities
```

Out[78]:

	asciiname	country code	alternatenames	latitude	longitude
0	'Abasan al Kabirah	PS	[Abasan, Abasan Kabera, Abasan al Kabir, Abasa...	31.31913	34.34005
1	'Afak	IQ	[Afag, Afak, Afaq, Sok el Afej, `Afak, Афак, ع...	32.06430	45.24743
2	'Ain Abid	DZ	['Ain Abid, Aine Abid, Aïne Abid, 'Aïn Abid]	36.23194	6.94333
3	'Ain Azel	DZ	['Ain Azel, Ajn-Azel', Ampere, Ampère, Айн-Азе...	35.81846	5.51115
4	'Ain Benian	DZ	['Ain Benian, Ain Berrian, Bou Ismail, Castigl...	36.80277	2.92185
...	...	...	...	...	...
28880	la Vila de Gracia	ES	[]	41.40315	2.15687
28881	les Escaldes	AD	[Ehskal'des-Ehndzhordani, Escaldes, Escaldes-E...	42.50729	1.53414
28882	les Roquetes	ES	[]	41.44809	2.17519
28883	maalot Tarshiha	IL	[Ma'alot-Tarshiha, Maalot Tarshiha, Maalot-Tar...	33.01667	35.26667
28884	thi xa Quang Tri	VN	[Quang Tri, Quảng Tri]	16.75083	107.18278

28885 rows × 5 columns

Da die Aggregation der Daten mehrstufig ist, dient die folgende Funktion der Ermittlung, wieviele Datensätze bereits angereicht wurden.

Hierzu berechnet sie den prozentualen Anteil der Datensätze in `df`, für die für alle in `cols` spezifizierten Spalten numerische Werte vorliegen.

```
In [79]: def print_percentage_of_filled_values(df, cols):
value = float(df[cols].notna().prod("columns").value_counts()[1]/df.shape[0])
print(f"{value:.2f}% of data sets have {" and ".join(cols)} data.")
```

Die erste Aggregationsstufe ist ein pandas merge über die Namen der Städte und die Länderkürzel.

`validate="one_to_one"` stellt sicher, dass die Anzahl der Datensätze durch den Merge gleich bleibt.

Hiermit können 92.75% der Datensätze mit Geokoordinaten angereichert werden.

```
In [80]: df = df.merge(  
    cities,  
    how="left",  
    left_on=["city", "alpha2"],  
    right_on=["asciiiname", "country code"],  
    validate="one_to_one",  
).drop(["asciiiname", "country code"], axis="columns")  
  
#df[["latitude", "longitude"]].isna().sum() # check if there are missing val  
df
```

Out[80]:

	city	country	meal_inexpensive	meal_mid_range	mcmeal	domestic_be
0	Delhi	India	4.90	22.04	4.28	
1	Shanghai	China	5.59	40.51	5.59	
2	Jakarta	Indonesia	2.54	22.25	3.50	
3	Manila	Philippines	3.54	27.40	3.54	
4	Seoul	South Korea	7.16	52.77	6.03	
...	...	...	...	...	...	
905	Haugesund	Norway	15.37	80.43	12.06	
906	Lagos	Portugal	8.31	62.32	7.27	
907	Porto Calvo	Brazil	4.66	25.16	5.59	
908	Fairfax	United States	19.38	75.00	10.00	
909	Kalach-na-Donu	Russia	8.52	49.13	6.14	

910 rows × 62 columns

Im nächsten Schritt wird von der Spalte "alternatenames" des DataFrames cities Gebrauch gemacht. Diese enthält eine Liste alternative Namen der jeweiligen Stadt.

In der folgenden Zeile wird für jede Stadt in df1, in der noch mindestens ein Wert fehlt, überprüft, ob deren Name als alternativer Name einer Stadt in cities auftaucht. Ist dem so, werden die Geokoordinaten eingetragen. So kann der Anteil der gefüllten Datensätze auf 98.35% erhöht werden.

```
In [81]: def get_nearest_match(row):
    alternate_matches_index = cities["alternatenames"].apply(lambda x: row["alternatenames"].lower() in x.lower())
    if not alternate_matches_index.sum() > 0:
        return row
    first_match = cities[alternate_matches_index].reset_index(drop=True).loc[alternate_matches_index.idxmin()]
    row["latitude"], row["longitude"] = first_match["latitude"], first_match["longitude"]
    return row

currently_no_value = df[["latitude", "longitude"]].notna().prod("columns")
df.loc[currently_no_value] = df.loc[currently_no_value].apply(get_nearest_match, axis=1)
print_percentage_of_filled_values(df, ["latitude", "longitude"])
```

98.35% of data sets have latitude and longitude data.

Die Anzahl der noch offenen Datensätze ist überschaubar, weshalb die Geokoordinaten zu diesen manuell recherchiert wurden (Quelle: [Wikipedia](#)). Über den Folgenden Code können somit auch die letzten Datensätze angereichert und die Aggregation mit Geokoordinaten somit abgeschlossen werden.

Höhenmeter

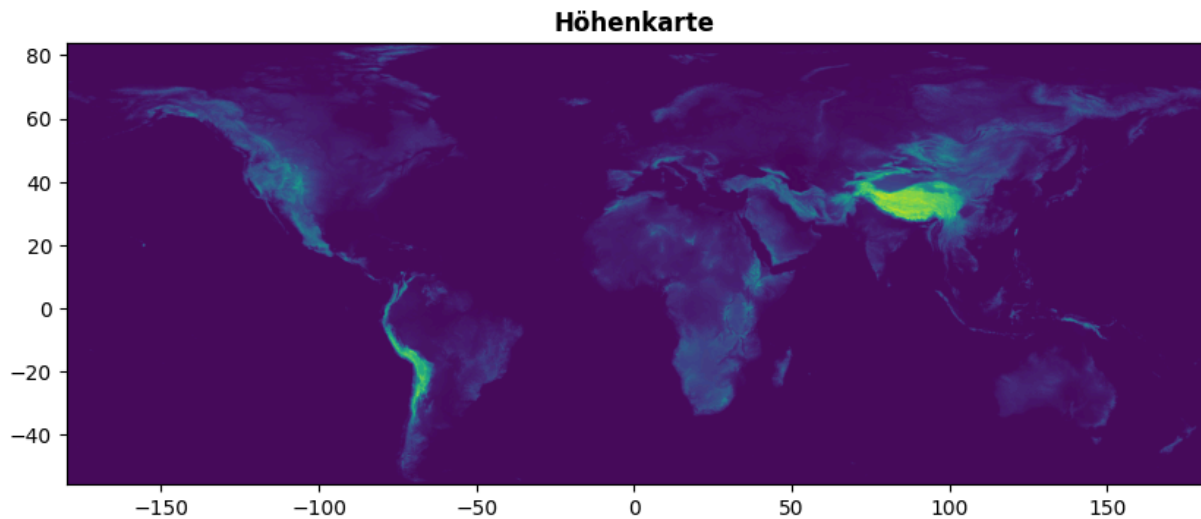
```
In [139]: import matplotlib.pyplot as plt

import rasterio
from rasterio.plot import show

raster_datei="data/GMTED2010_averaged.tif"

with rasterio.open(raster_datei) as src:
    fig, ax = plt.subplots(figsize=(10, 10))
    show(src, ax=ax, title="Höhenkarte")
    plt.show()

    # Ausgabe der Auflösung in Bogensekunden (arc seconds)
    resolution = src.res[0] # Annahme: quadratische Pixel
    arc_sec = resolution # Umrechnung von Grad in Bogensekunden
    print(f"Auflösung: {arc_sec:.2f} arc seconds")
```



Auflösung: 0.25 arc seconds

In [83]: `df.count()`

```
Out[83]: city          910
country        910
meal_inexpensive  910
meal_mid_range   910
mcmeal          910
...
alpha2          910
continent       910
alternatenames   844
latitude        895
longitude        895
Length: 62, dtype: int64
```

```
In [ ]: from rasterio.transform import rowcol
import numpy as np

def get_height_at_coordinates(longitude, latitude, raster_file="data/GMTED20
    """
    Gibt die Höhe für gegebene geografische Koordinaten (Longitude, Latitude

    :param longitude: Longitude-Wert (Längengrad)
    :param latitude: Latitude-Wert (Breitengrad)
    :param raster_file: Pfad zur Rasterdatei
    :return: Höhe an den angegebenen Koordinaten
    """

    with rasterio.open(raster_file) as src:
        # Transformiere die geografischen Koordinaten in Raster-Koordinaten
        row, col = rowcol(src.transform, longitude, latitude)

        # Hole die Höhendaten als numpy array
        height_data = src.read(1) # Liest das erste Band (Höhendaten)

        # Hole den Wert aus den Rasterdaten
```



```

        value = height_data[row, col]
        return value

# Beispielnutzung der Funktion
latitude, longitude = 48.782624819735624, 9.167036756166327 # Stuttgart
höhe = get_height_at_coordinates(longitude, latitude) # Der Wert ist die du
print(f"Höhe an der Position ({longitude}, {latitude}): {höhe}")

```

Höhe an der Position (9.167036756166327, 48.782624819735624): 294

```

In [85]: not_na = ~df[["latitude", "longitude"]].isna().any(axis=1)
df["elevation"] = np.nan
df.loc[not_na, "elevation"] = df[not_na].apply(
    lambda row: get_height_at_coordinates(row["longitude"], row["latitude"])
    axis=1
)

```

Andere Ausreiser

Data Exploration

```

In [86]: # Warenkorb

# Define a weighted basket for a person in the creative industry
basket_weights = {
    'meal_inexpensive': 25,
    'meal_mid_range': 3,
    'mcmeal': 3,
    'domestic_beer_restaurant': 2,
    'imported_beer_restaurant': 2,
    'cappuccino': 8,
    'coke': 5,
    'water_033': 5,
    'milk': 4,
    'white_bread': 3,
    'rice': 1,
    'eggs': 1,
    'cheese': 1,
    'chicken': 3,
    'beef': 2,
    'apples': 1,
    'banana': 1.25,
    'oranges': 2,
    'tomato': 1,
    'potato': 0.5,
    'onion': 0.25,
    'lettuce': 4,
    'water_150': 3,
    'wine': 2,
    'domestic_beer_market': 3,
    'imported_beer_market': 3,
    'cigarettes': 0,
    'one_way_ticket': 0,
}

```

```

    'monthly_pass': 1,
    'taxi_start': 3,
    'taxi_1km': 7,
    'taxi_1h': 2,
    'gasoline': 0,
    'vw_golf': 0,
    'toyota_corolla': 0,
    'basic_85m2': 0.75,
    '1min_prepaid': 50,
    'internet': 1,
    'fitness_club': 1,
    'tennis': 1,
    'cinema': 1,
    'preschool': 0,
    'primary_school': 0,
    'jeans': 0.75,
    'summer_dress': 0.75,
    'nike_shoes': 0.75,
    'business_shoes': 0.125,
    'apartment_1br_center': 1,
    'apartment_1br_outside': 0,
    'apartment_3br_center': 0,
    'apartment_3br_outside': 0,
    'per_square_meter_center': 0,
    'per_square_meter_outside': 0,
    'avg_month_salary': 0,
    'mortgage_interest': 0
}

```

```

# Apply the basket_weights dictionary to the dataframe
df["weighted_basket"] = df[list(basket_weights.keys())].mul(basket_weights).

df[["city", "country", "continent", "weighted_basket"]]

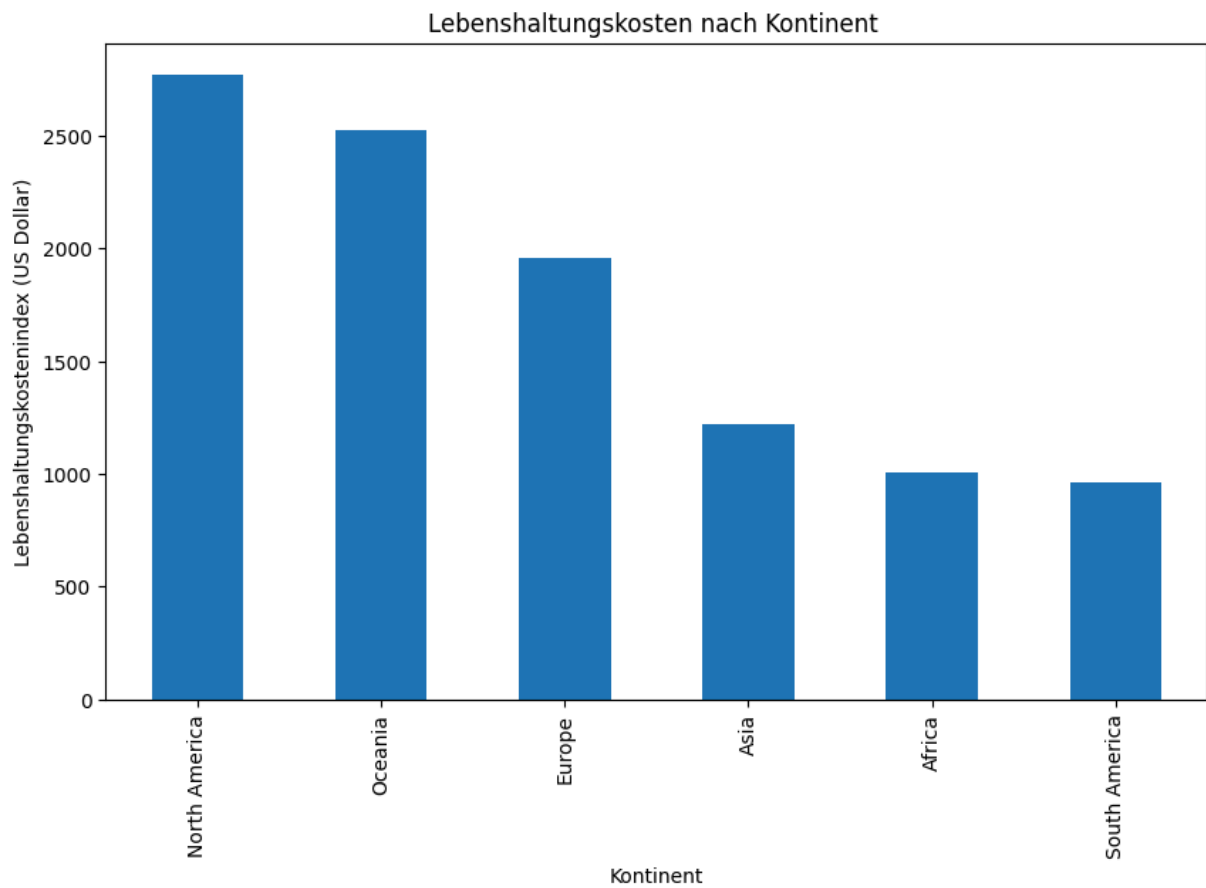
```

Out[86]:

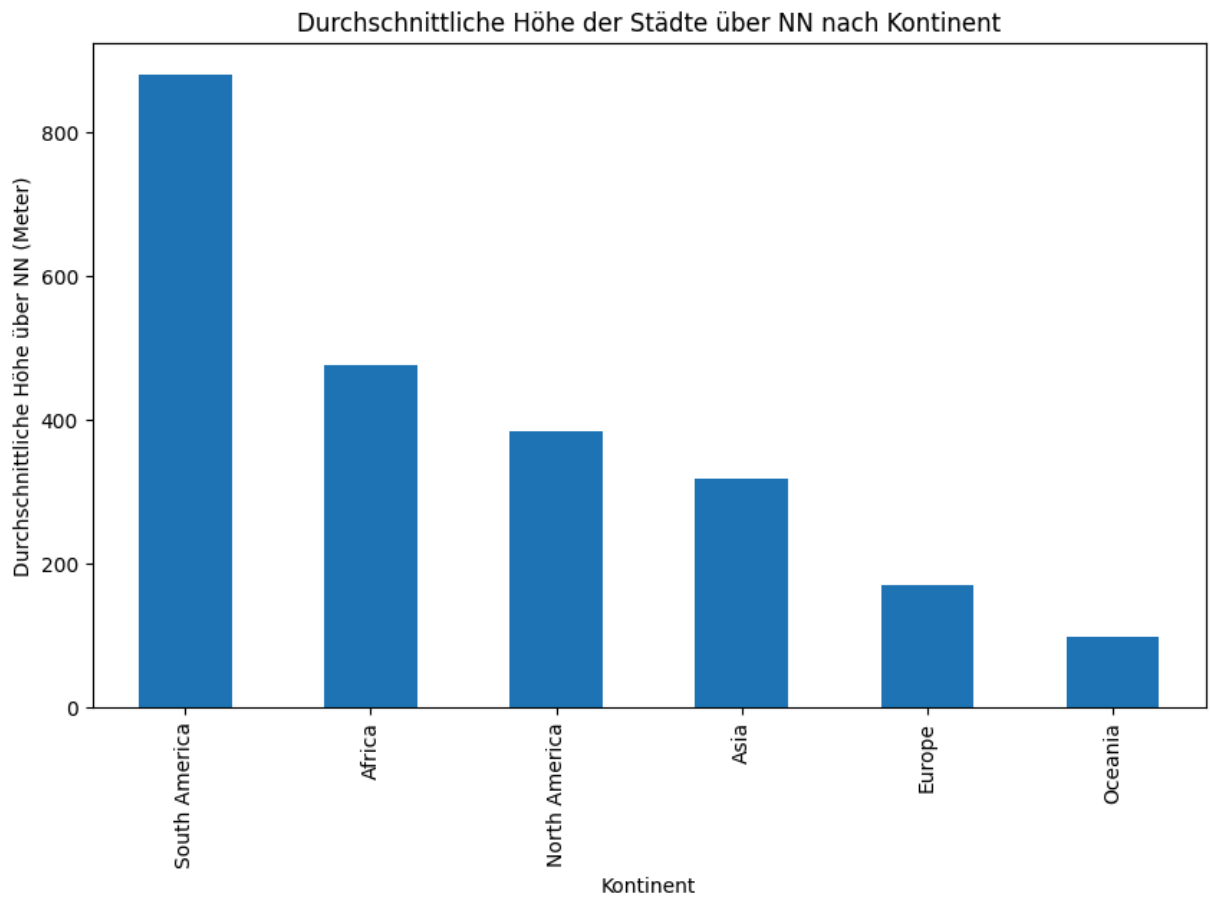
	city	country	continent	weighted_basket
0	Delhi	India	Asia	722.28500
1	Shanghai	China	Asia	1900.77625
2	Jakarta	Indonesia	Asia	1048.52250
3	Manila	Philippines	Asia	1174.28750
4	Seoul	South Korea	Asia	1922.65250
...	...	...	...	...
905	Haugesund	Norway	Europe	2319.10750
906	Lagos	Portugal	Europe	1845.88500
907	Porto Calvo	Brazil	South America	941.85875
908	Fairfax	United States	North America	3694.21500
909	Kalach-na-Donu	Russia	Europe	1439.86000

910 rows × 4 columns

```
In [87]: ax = df.groupby("continent")["weighted_basket"].mean().sort_values(ascending
        kind="bar", figsize=(10, 6)
        )
ax.set_ylabel("Lebenshaltungskostenindex (US Dollar)")
ax.set_xlabel("Kontinent")
ax.set_title("Lebenshaltungskosten nach Kontinent ")
plt.show()
```

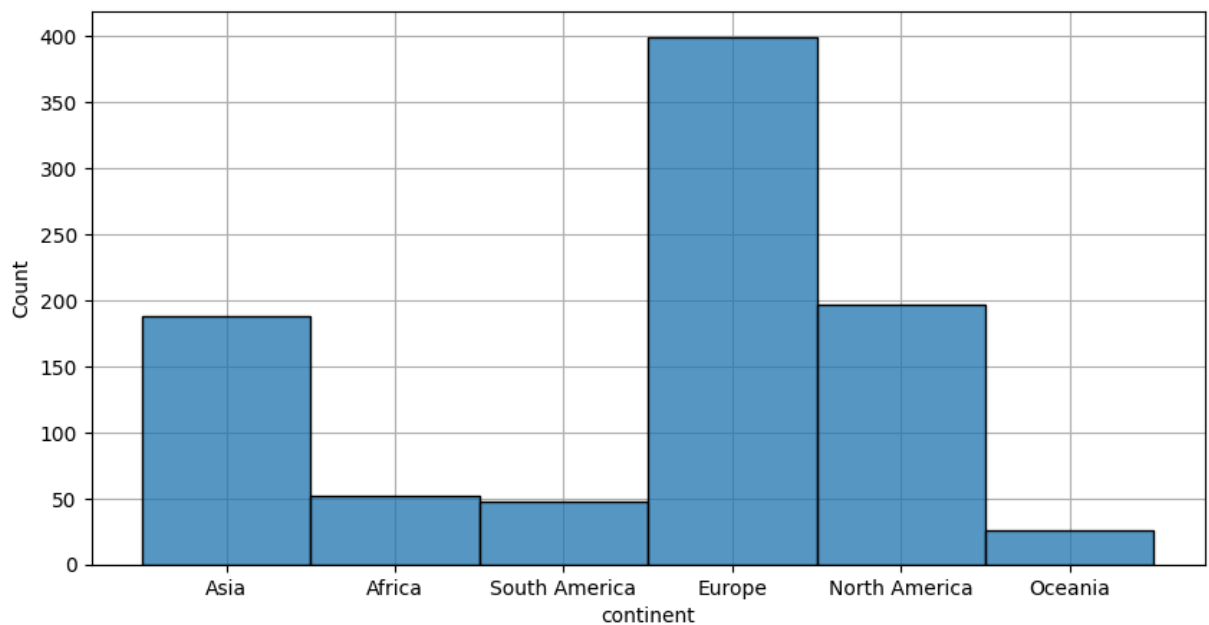


```
In [88]: ax = df.groupby("continent")["elevation"].mean().sort_values(ascending=False,
    kind="bar", figsize=(10, 6))
ax.set_ylabel("Durchschnittliche Höhe über NN (Meter)")
ax.set_xlabel("Kontinent")
ax.set_title("Durchschnittliche Höhe der Städte über NN nach Kontinent")
plt.show()
```



Verteilung auf die Kontinente

```
In [89]: fig, ax = plt.subplots(1, 1, figsize=(10,5))
sns.histplot(df["continent"])
ax.set_axisbelow(True)
plt.grid()
```



Verteilung auf Karte

```
In [90]: import plotly.express as px

fig = px.scatter_geo(
    df,
    lat="latitude",
    lon="longitude",
    size="weighted_basket",
    color="country",
    #symbol="continent",
    hover_name="city",
    hover_data=["country", "elevation", "weighted_basket"],
    projection="natural earth",
    height=600,
    width=1000,
)
fig.update_layout(showlegend=False)
```

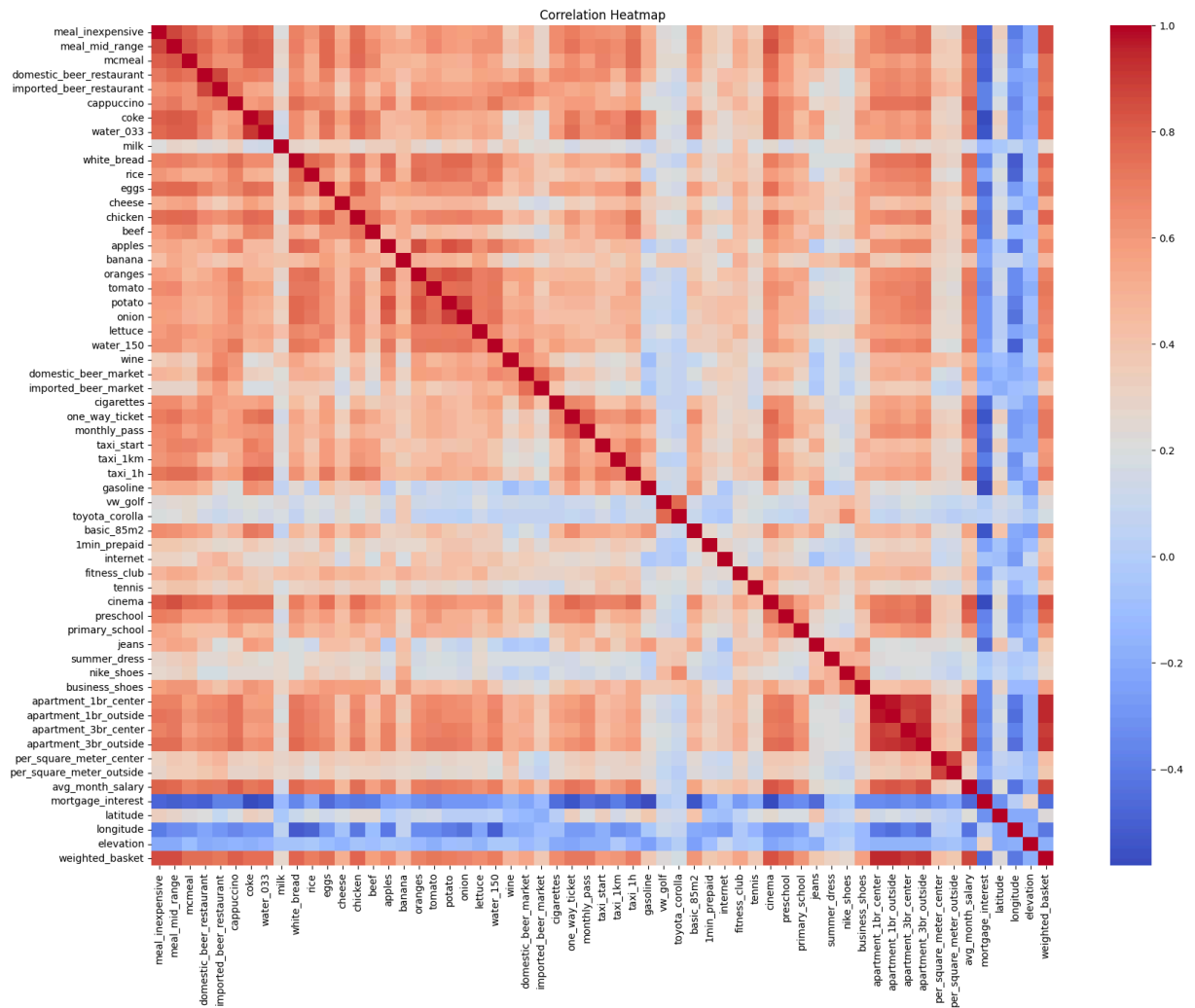
Korrelationsmatrix

```
In [91]: import matplotlib.pyplot as plt
import seaborn as sns

numeric_columns = df.select_dtypes(np.number)

# Compute correlation matrix
correlation_matrix = numeric_columns.corr()

# Plot heatmap with all column names displayed
plt.figure(figsize=(20, 15))
sns.heatmap(correlation_matrix, annot=False, cmap="coolwarm", fmt=".2f", xti
plt.title("Correlation Heatmap")
plt.xticks(rotation=90)
plt.yticks(rotation=0)
plt.show()
```



```
In [92]: df.count().sort_values(ascending=True)
```

```
Out[92]: primary_school      834
alternatenames      844
1min_prepaid      886
per_square_meter_outside      886
taxi_1h      887
...
apartment_3br_center      910
business_shoes      910
alpha2      910
continent      910
weighted_basket      910
Length: 64, dtype: int64
```

Städte Cluster

```
In [ ]: from sklearn.cluster import KMeans, SpectralClustering, MiniBatchKMeans, Bis
from sklearn.metrics import silhouette_score, calinski_harabasz_score, davie
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans, AgglomerativeClustering
```

```

from sklearn.preprocessing import StandardScaler

import pandas as pd

# Selektiere numerische Spalten
numeric_columns = df.select_dtypes(np.number)

# Skalieren die Daten
scaler = StandardScaler()
scaled_numeric = pd.DataFrame(scaler.fit_transform(numeric_columns))

not_na = ~numeric_columns.isna().any(axis=1)
data_to_cluster = scaled_numeric.dropna()
# Initialize results storage
results = []

# Define clustering methods
clustering_methods = {
    "KMeans": lambda n_clusters: KMeans(n_clusters=n_clusters, random_state=
    "Agglomerative": lambda n_clusters: AgglomerativeClustering(n_clusters=n
    "SpectralClustering": lambda n_clusters: SpectralClustering(n_clusters=n
    "BisectingKMeans": lambda n_clusters: BisectingKMeans(n_clusters=n_clust
}

# Cluster sizes to evaluate
cluster_sizes = list(range(2, 40, 2))

cluster_dict = {}

# Perform clustering for each method and cluster size
for method_name, method in clustering_methods.items():
    for n_clusters in cluster_sizes:

        iterations = 1
        if method_name in ["KMeans", "MiniBatchKMeans", "BisectingKMeans"]:
            iterations = 10
        for i in range(iterations):
            clusters = method(n_clusters).fit_predict(data_to_cluster)

            cluster_dict[(method_name, n_clusters)] = clusters

            silhouette_avg = silhouette_score(data_to_cluster, clusters)
            calinski_harabasz = calinski_harabasz_score(data_to_cluster, clu
            davies_bouldin = davies_bouldin_score(data_to_cluster, clusters)
            results.append({
                "Method": method_name,
                "Clusters": n_clusters,
                "Silhouette Score": silhouette_avg,
                "Calinski-Harabasz Index": calinski_harabasz,
                "Davies-Bouldin Index": davies_bouldin
            })

# Create a DataFrame with a MultiIndex
metrics_df = pd.DataFrame(results)

```



```
metrics_df = metrics_df.groupby(["Method", "Clusters"]).mean().reset_index()

metrics_df.set_index(["Method", "Clusters"], inplace=True)
metrics_df.sort_index(inplace=True)
```

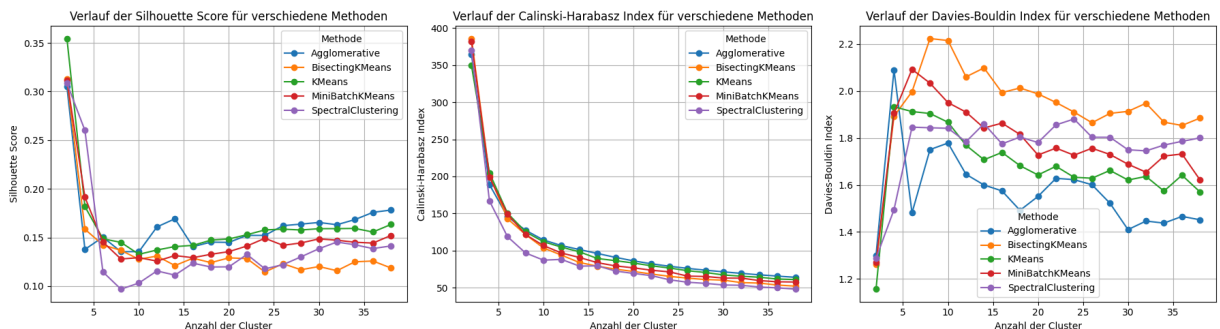
```
In [96]: import matplotlib.pyplot as plt

# Erstelle eine Liste der Metriken
metrics = ["Silhouette Score", "Calinski-Harabasz Index", "Davies-Bouldin In

# Erstelle eine Figure mit Subplots
fig, axes = plt.subplots(1, len(metrics), figsize=(18, 5), sharex=True)

# Iteriere über die Metriken und erstelle für jede einen separaten Plot
for i, metric in enumerate(metrics):
    ax = axes[i]
    for method in metrics_df.index.get_level_values("Method").unique():
        subset = metrics_df.loc[method]
        ax.plot(
            subset.index,
            subset[metric],
            marker="o",
            label=method
        )
    ax.set_title(f"Verlauf der {metric} für verschiedene Methoden")
    ax.set_xlabel("Anzahl der Cluster")
    ax.set_ylabel(metric)
    ax.legend(title="Methode")
    ax.grid(True)

# Passe das Layout an
plt.tight_layout()
plt.show()
```



```
In [97]: def visualize_clusters(data_to_cluster, clusters, plotly=False, data_names=N

# Führe PCA durch
pca = PCA(n_components=2)
pca_result = pca.fit_transform(data_to_cluster.dropna())

if plotly:
    # Visualisierung mit Plotly
    fig = px.scatter(
        x=pca_result[:, 0],
```

```

        y=pca_result[:, 1],
        color=clusters,
        labels={"x": "PCA 1", "y": "PCA 2"},
        hover_name=data_names,
        title="Cluster Visualization with PCA",
        height=700,
        width=1100,
    )
    fig.show()
    return

# Visualisierung mit Matplotlib
fig, ax = plt.subplots(figsize=(10, 7))
scatter = ax.scatter(
    pca_result[:, 0],
    pca_result[:, 1],
    c=clusters[~data_to_cluster.isna().any(axis=1)],
    cmap='viridis',
    alpha=0.7
)
ax.set_xlabel("t-SNE 1")
ax.set_ylabel("t-SNE 2")
ax.set_title("Cluster Visualization")
plt.colorbar(scatter, ax=ax, label="Cluster")
plt.show()

```

```
visualize_clusters(data_to_cluster, cluster_dict[("KMeans", 10)], plotly=True)
```

4. Applepreis Vorhersage

Der Applepreis wird anhand vorhandener Features im Datensatz vorhergesagt. Dazu wird zunächst eine Feature-Selektion durchgeführt, und anschließend ein Estimator verwendet, um den Preis zu prognostizieren. Um einen Überblick zu gewinnen, werden verschiedene Selektionsverfahren, Estimatoren sowie unterschiedliche Anzahlen an Features getestet. Jedes Ergebnis (R^2 und RMSE) wird im Rahmen einer 5-fachen Kreuzvalidierung ermittelt.

Wichtig: Wenn die Feature-Auswahl beispielsweise auf der Korrelation zwischen dem Applepreis und den anderen Features basiert, werden für die Korrelationsberechnung ausschließlich die Trainingsdaten verwendet. Alles andere würde zu einer Vermischung von Trainings- und Testdaten führen – denn auch die Feature-Auswahl stellt eine Trainingsentscheidung dar. Im Code wird dies dadurch sichergestellt, dass die gesamte Pipeline aus Selektion und Estimator in die Kreuzvalidierung eingebunden wird – und nicht nur der Estimator allein.

```
In [ ]: from sklearn.svm import SVR
        from sklearn.model_selection import train_test_split
```

```

from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.ensemble import RandomForestRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.feature_selection import SelectKBest, f_regression, mutual_info
from sklearn.pipeline import Pipeline

n_values = list(range(1, 20)) # Verschiedene Werte von n, über die iteriert

# Initialisiere Speicher für Ergebnisse
results = []

# Bewertungsfunktion für die Auswahl von Features anhand der Korrelation mit
def compute_correlation(X, y):
    """Berechnet die Korrelation zwischen Features und Zielwert.
    Argumente:
        X: ndarray, Form (n_samples, n_features)
        y: Zielvariable
    Rückgabe:
        ndarray, Form (n_features,)
        Korrelationskoeffizienten zwischen jedem Feature und der Zielvar
    """
    # Numpy-Trick: Die Funktion gibt eigentlich eine Matrix zurück, die letz
    # enthält die gewünschten Koeffizienten
    return np.abs(np.corrcoef(X.T, y)[-1, :-1])

x = numeric_columns.drop(columns=["apples"]).dropna()
y = numeric_columns["apples"].loc[x.index]

for n in n_values:

    # Definiere Methoden zur Feature-Auswahl
    feature_selection_methods = {
        "f_regression": SelectKBest(score_func=f_regression, k=n),
        "mutual_info": SelectKBest(score_func=mutual_info_regression, k=n),
        "correlation": SelectKBest(score_func=compute_correlation, k=n),
    }

    # Definiere Modelle
    base_estimators = {
        "Lineare Regression": LinearRegression(),
        "SVR": SVR(kernel="rbf"),
        "Random Forest": RandomForestRegressor(n_estimators=100),
        "K-Nearest Neighbors": KNeighborsRegressor(n_neighbors=5),
    }

    # Erstelle Pipelines für alle Kombinationen aus Feature-Auswahl und Mode
    estimators = {
        (fs_name, est_name): Pipeline([
            ("feature_selection", fs_method),
            ("estimator", estimator)
        ])
        for fs_name, fs_method in feature_selection_methods.items()
        for est_name, estimator in base_estimators.items()
    }

```

```

    }

    # Trainiere und bewerte jedes Modell
    for name, estimator in estimators.items():
        scores_r2 = cross_val_score(estimator, x, y, cv=5, scoring="r2")
        scores_rmse = np.sqrt(-cross_val_score(estimator, x, y, cv=5, scoring="rmse"))

        results.append({
            "n": n,
            "Feature-Auswahl": name[0],
            "Modell": name[1],
            "RMSE": scores_rmse.mean(),
            "R²": scores_r2.mean(),
            "RMSE Std": scores_rmse.std(),
            "R² Std": scores_r2.std()
        })

# Ergebnisse mit MultiIndex anzeigen
results_df = pd.DataFrame(results).set_index(["Modell", "Feature-Auswahl"])

```

Out[]:

		n	RMSE	R²	RMSE Std	R² Std
Modell Feature-Auswahl						
Lineare Regression	f_regression	1	0.724122	0.705584	0.046299	0.045169
SVR	f_regression	1	0.793919	0.658745	0.164595	0.058127
Random Forest	f_regression	1	0.849844	0.602202	0.091969	0.026490
K-Nearest Neighbors	f_regression	1	0.819772	0.632469	0.113561	0.021349
Lineare Regression	mutual_info	1	0.724122	0.705584	0.046299	0.045169
...	...	...	...	...	...	...
K-Nearest Neighbors	mutual_info	19	0.933751	0.525417	0.185836	0.090914
Lineare Regression	correlation	19	0.641410	0.765023	0.086125	0.068282
SVR	correlation	19	0.959357	0.497331	0.207832	0.116164
Random Forest	correlation	19	0.602110	0.803110	0.155781	0.046111
K-Nearest Neighbors	correlation	19	0.963998	0.493795	0.168085	0.071086

228 rows × 7 columns

Hinweis: Vor allem bei der Selektion zeigten sich univariate Verfahren wie beispielsweise Forward- oder Backward-Selektion als wenig vielversprechend, weswegen diese in unserem Vergleich nicht weiter berücksichtigt wurden. Für eine Auswertung anderer Estimatoren und Selektionsverfahren muss lediglich in den Dictionaries `feature_selection_methods` oder `base_estimators` eine Zeile ergänzt und ein entsprechendes `sklearn`-Objekt übergeben werden.

4.1 Ergebnisse

Die folgenden Plots zeigen den Verlauf der Metriken für die einzelnen Kombinationen aus Estimator und Selektionsverfahren. Kombinationen, die eine sehr schwache Performance gezeigt haben – darunter die Ansätze mit SVR und K-Nearest Neighbors – wurden aus Gründen der Übersichtlichkeit entfernt.

```
In [165... import matplotlib.pyplot as plt

# Erstelle eine Liste der Metriken
metrics = ["RMSE", "R2"]

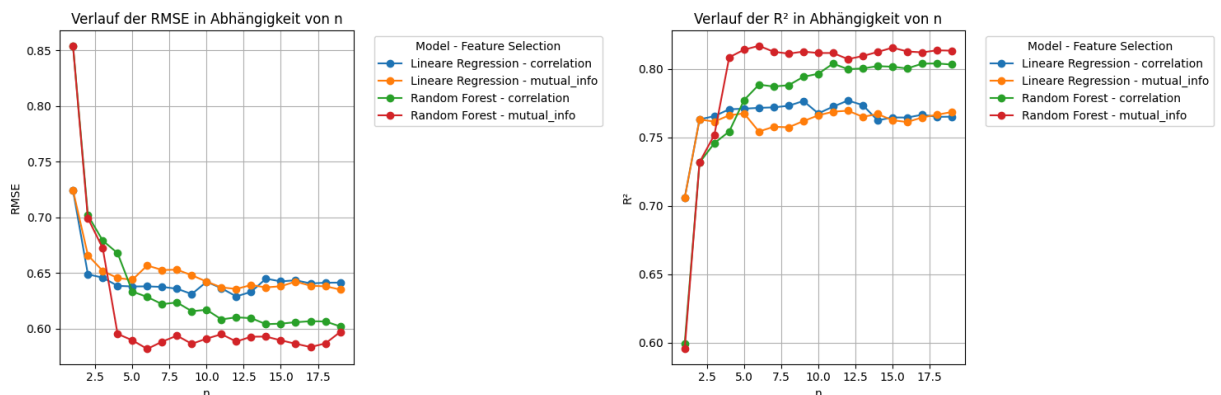
# Erstelle eine Figure mit Subplots
fig, axes = plt.subplots(1, len(metrics), figsize=(15, 5), sharex=True)

# Iteriere über die Metriken und erstelle für jede einen separaten Plot
for i, metric in enumerate(metrics):
    ax = axes[i]
    for (model, feature_selection), subset in results_df.groupby(["Modell",

        if model in ["SVR", "K-Nearest Neighbors"]:
            continue
        if feature_selection in ["f_regression"]:
            continue

    ax.plot(
        subset["n"],
        subset[metric],
        marker="o",
        label=f"{model} - {feature_selection}"
    )
    ax.set_title(f"Verlauf der {metric} in Abhängigkeit von n")
    ax.set_xlabel("n")
    ax.set_ylabel(metric)
    ax.legend(title="Model - Feature Selection", bbox_to_anchor=(1.05, 1), 1
    ax.grid(True)

# Passe das Layout an
plt.tight_layout()
plt.show()
```



Varianzbetrachtung

```
In [168... results_df[results_df["n"] == 6].groupby(["Modell", "Feature-Auswahl"]).mean
```

Out[168...

		n	RMSE	R ²	RMSE Std	R ² Std
Modell	Feature-Auswahl					
Random Forest	mutual_info	6.0	0.581864	0.816823	0.145700	0.043888
	f_regression	6.0	0.621414	0.789069	0.121648	0.033351
	correlation	6.0	0.628517	0.788432	0.116985	0.029523
Lineare Regression	correlation	6.0	0.638042	0.771485	0.079501	0.045016
	f_regression	6.0	0.638042	0.771485	0.079501	0.045016
K-Nearest Neighbors	correlation	6.0	0.654643	0.767548	0.121408	0.031221
	f_regression	6.0	0.654643	0.767548	0.121408	0.031221
Lineare Regression	mutual_info	6.0	0.656816	0.754195	0.080701	0.066780
SVR	f_regression	6.0	0.676175	0.753589	0.182999	0.070396
	correlation	6.0	0.676175	0.753589	0.182999	0.070396
K-Nearest Neighbors	mutual_info	6.0	0.719171	0.720572	0.166054	0.059062
SVR	mutual_info	6.0	0.803790	0.676549	0.173703	0.075589

4.2 Fazit

Bei der Anzahl der Features fällt auf, dass bereits eine kleine Auswahl (etwa sechs Features) ausreicht, um eine Vorhersage zu treffen, die sich durch zusätzliche Features nicht weiter verbessern lässt. Dies lässt sich grundsätzlich damit erklären, dass nur ein geringer Teil der Features tatsächlich eine starke Korrelation mit dem Apfelpreis aufweist.

Zwar konnte mit dem Random-Forest-Ansatz ein besseres Ergebnis als mit der linearen Regression erzielt werden, insgesamt unterscheidet sich die Performance der Modelle jedoch nicht signifikant.

Generell zeigt sich beim Random-Forest-Ansatz (n = 6) eine höhere Standardabweichung der Metriken über die einzelnen Folds hinweg im Vergleich zur linearen Regression. Letztere weist in Bezug auf den RMSE-Score tendenziell eine

geringere Verzerrung auf, ist jedoch insgesamt ungenauer. Der R^2 -Wert zeigt hingegen ein umgekehrtes Verhalten.

Insgesamt lässt sich anhand dieses Beispiels kein klarer Trend in Bezug auf das Bias-Variance-Trade-off zwischen den Verfahren erkennen (auch nicht in Bezug auf die Anderen Verfahren).